

Task 19: Handling Null Values in Pandas

Track: Data Science | Level 1 - Day 19 | Tools: Python, Pandas, Jupyter Notebook

Level 1 - Day 19: Handling Null Values & Advanced Imputation in Pandas

Track: Data Science

Task: Task 19 - Advanced Null Value Profiling, Imputation Strategies, and Data Cleaning

Objective: Master comprehensive detection, profiling, and remediation of null values (`NaN`, `None`, ``) in Pandas, including forward/backward fills (`ffill`/`bfill`), polynomial & linear interpolation, group-wise conditional imputation (`transform`), and tracking missingness indicators.

1. Quick Reference: Null Value Patterns & Methods in Pandas

Method / Technique	Syntax Pattern	Best Used For
Null Profiling	<code>df.isnull().sum()</code> , <code>df.info()</code>	Initial assessment of completeness across columns
Sequential Fill	<code>df['col'].ffill()</code> , <code>df['col'].bfill()</code>	Time-series, telemetry, and ordered sensor readings
Interpolation	<code>df['col'].interpolate(method='linear')</code>	Continuous numerical series with gradual trends
Group-wise Imputation	<code>df.groupby('dept')['sal'].transform('median')</code>	Segment-specific central tendencies (e.g., salary by department)
Missingness Indicator	<code>df['col_was_missing'] = df['col'].isnull().astype(int)</code>	Preserving information about missingness for ML models
Selective Replace	<code>df.replace({-999: np.nan, '?': np.nan})</code>	Converting custom placeholder sentinels to standard `NaN`

2. Generating a Realistic Multi-Type Dataset with Null Sentinels

In [code]

```
import pandas as pd
import numpy as np

# Simulating realistic enterprise customer & transaction dataset with various null types
data = {
    'Transaction_ID': ['TX1001', 'TX1002', 'TX1003', 'TX1004', 'TX1005', 'TX1006', 'TX1007', 'TX1008'],
    'Customer_Name': ['Aman', 'Pooja', 'Rohan', 'Sneha', 'Vikas', 'Aditi', 'Karan', 'Neha'],
    'Department': ['Tech', 'Finance', 'Tech', 'HR', 'Finance', 'Tech', 'HR', 'Finance'],
    'Salary': [95000.0, np.nan, 88000.0, 52000.0, np.nan, 120000.0, 50000.0, 92000.0],
    'Credit_Score': [740, -999, 680, 710, -999, 790, 640, -999], # -999 sentinel placeholder
    'Daily_Stock_Trend': [150.2, 152.0, np.nan, np.nan, 158.4, 160.1, np.nan, 165.0],
    'Approval_Status': ['Approved', 'Pending', np.nan, 'Approved', 'Rejected', np.nan, 'Approved',
    'Approved']
}

df = pd.DataFrame(data)
print("=== RAW DATASET SNAPSHOT ===")
df
```

3. Sentinel Replacement & Null Heatmap Profiling

In [code]

```
# Convert sentinel placeholder values (-999) to true numpy NaNs
df['Credit_Score'] = df['Credit_Score'].replace(-999, np.nan)

# Comprehensive null profile audit
null_counts = df.isnull().sum()
null_percentages = (df.isnull().mean() * 100).round(2)

null_audit_table = pd.DataFrame({
    'Column_Name': df.columns,
    'Missing_Records': null_counts.values,
    'Null_Percentage (%)': null_percentages.values,
    'Data_Type': df.dtypes.values
})

print("=== COMPREHENSIVE NULL AUDIT REPORT ===")
null_audit_table
```

4. Advanced Imputation Techniques

1. **Linear Interpolation:** Reconstructing smooth sequential trajectories in `Daily\_Stock\_Trend`.
2. **Group-wise Conditional Imputation:** Imputing missing `Salary` using departmental medians.
3. **Missingness Indicator Flagging:** Preserving whether `Credit\_Score` was originally absent.
4. **Forward/Backward Fill (`ffill`/`bfill`):** Cascading categorical approvals.

In [code]

```
df_clean = df.copy()

# 1. Track missingness indicator before modifying Credit_Score
df_clean['Credit_Score_Was_Missing'] = df_clean['Credit_Score'].isnull().astype(int)

# Impute Credit_Score with overall median
df_clean['Credit_Score'] = df_clean['Credit_Score'].fillna(df_clean['Credit_Score'].median())

# 2. Sequential Linear Interpolation for trend metrics
df_clean['Daily_Stock_Trend'] = df_clean['Daily_Stock_Trend'].interpolate(method='linear').round(2)

# 3. Group-wise conditional imputation: Salary by Department median
dept_median_salaries = df_clean.groupby('Department')['Salary'].transform('median')
df_clean['Salary'] = df_clean['Salary'].fillna(dept_median_salaries)

# 4. Forward fill remaining categorical values
df_clean['Approval_Status'] = df_clean['Approval_Status'].ffill().bfill()

print("=== FULLY CLEANED DATASET (0 NULLS) ===")
print(df_clean)
print(f"\nRemaining Null Values Total: {df_clean.isnull().sum().sum()}")
```

5. Applied Business Metrics on Cleaned Data

In [code]

```
# Compute risk adjustment factor and adjusted compensation
df_clean['Risk_Tier'] = np.where(
    df_clean['Credit_Score'] >= 720, 'Prime (Low Risk)',
    np.where(df_clean['Credit_Score'] >= 660, 'Near Prime', 'Subprime')
)

print("=== FINAL ENRICHED DATASET SUMMARY ===")
df_clean[['Transaction_ID', 'Customer_Name', 'Department', 'Salary', 'Credit_Score', 'Risk_Tier',
'Approval_Status']]
```